

Enxi Li & Yuchen Jiu
& Yan Yan & Alisa Fedorenko

Automated Ball Projection System Using Piezoelectric and Infrared Break Beam Sensors for Target Accuracy

Applied Control Project

Advisor

Prof. Dr. Georg Rauter

July 2024

Abstract

The project aims to tune the controller as much as possible to achieve very precise results and score as many baskets as possible. The initial project was done with TwinCAT3, it included different configuration parameters, which could be updated in Matlab.

To improve basketball shooting accuracy, we first optimized the parameters to ensure more shots successfully entered the hoop after the pick-up and launch processes. After optimization, we noticed instability in the system's performance. We found that the varying positions of the ball after each pick-up were a key issue. To address this, we added a correction track step in Simulink, which significantly improved shooting accuracy. Once the accuracy was maximized, we incorporated sensors to count the number of shots and enable automatic continuous shooting. Using piezoelectric and infrared breakbeam sensors, we successfully achieved our goal.

Finally, our 2-minute shooting accuracy and 10-shot accuracy both approached 100%, demonstrating the effectiveness of our efforts. Additionally, the scores displayed on the screen matched the recorded scores, confirming the success of our counting system.

Acknowledgements

I want to thank everyone who took part in this project for being helpful and responsible during the whole project. I want to thank Li Enxi for checking which sensors are more suitable in our case and finding two good options on the market. Yan Yan helped a lot with installing these sensors. Also, I want to thank Jiu Yuchen, Yan Yan, Li Enxi and Alisa Fedorenko for always being ready to come to the lab (even on weekends) to test their ideas and implement the best solution in the end. Jiu Yuchen showed patience and good understanding of the physics behind the system. Alisa Fedorenko could demonstrate knowledge from the block course: Hands-on Introduction to Medical Robotics Hardware. And of course, I want to thank everyone for writing their parts in the final report.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
2 Parameter	2
2.1 Initial Offset of the Plate	2
2.2 Picking Up	2
2.2.1 Basketball.P.c.PickUp1	2
2.2.2 Basketball.P.c-pickUp2	2
2.3 Launching	2
2.3.1 Basketball.P.c.launch1	2
2.3.2 Basketball.P.accelTime	3
2.3.3 Basketball.P.c.launch2	3
2.3.4 Basketball.P. vmax_AM8111_1F21	3
3 Stabilization	4
3.1 Process Overview	4
3.2 Ball Position Stabilization	4
3.3 Ideas About Automation	5
4 Sensor Configuration	6
4.1 Piezoelectric Sensor	6
4.1.1 Sensor Selection and Installation	6
4.1.2 Connection to device	6
4.1.3 Signal Processing	7
4.2 Infrared Breakbeam Sensor	8
4.2.1 Sensor Selection and Installation	8
4.2.2 Connection to device	9
4.2.3 Signal Processing	9
5 Results and Discussion	11

1 Introduction

The goal of the project was to improve the existing controller, which was working in TWINCAT3. Initially, it was a construction, which consisted of 3 main components: rotating pendulum, basket and ball.

The pendulum was rotating with specified rules, which could be configured in the programming code. It could pick up the ball and throw it into the basket (see Figure below 1.1). The goal was to tune the controller in a such way that the pendulum could score as many baskets as possible.

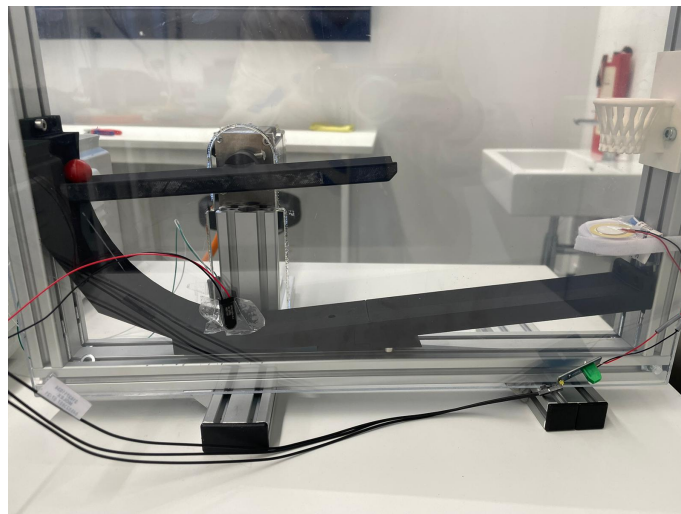


Figure 1.1: Project

During the project, we were working with Simulink and Matlab, where it was possible to change different parameters of the controller such as speed or torque of the pendulum. In the end, the goal was to understand how all these parameters should be defined in order to make the controller very accurate and precise so that the system could make the biggest number of baskets.

The efficiency of the controller was measured with 2 different methods:

1. Scoring baskets within 2 minutes
2. Scoring as many baskets as possible with 10 shots

In order to control the whole system and do it more precisely two sensors were added. The first sensor was piezoelectric and it measured the force, with which the ball hit the surface, and in such a way it counted each successful basket.

The second sensor is an infrared one. It checked if the ball was passing through a certain point in space. It gave additional control of the position of the ball.

Further, there is an explanation how exactly these sensors were installed and what changes were done in the code.

2 Parameter

The parameters can be categorized into three groups: initial offset of the plate, picking up the basketball, and launching the basketball.

2.1 Initial Offset of the Plate

After pressing the ‘gui.activateTiltControl’ button, the motor prepares for rotation. Once the motor torque angle reaches the predefined offset, it stops rotating clockwise continuously. We adjusted the ‘**Basketball.motorAnglePlateZeroOffset_Value**’ so that the plate can stop in a horizontal position.

2.2 Picking Up

2.2.1 Basketball_P.c_PickUp1

This is the angle by which the plate rotates counterclockwise from the horizontal position before picking up the ball (Fig 2.1). We slightly adjusted this angle to ensure that the plate’s counterclockwise rotation does not push the moving ball backward.



Figure 2.1: Pick Up Angle 1

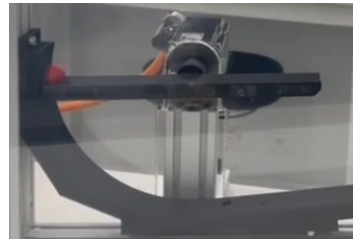


Figure 2.2: Pick Up Angle 2

2.2.2 Basketball_P.c_pickUp2

This is the angle at which the plate will stop after picking up the ball (Fig 2.2). Our goal is to bring this angle closer to the horizontal position.

2.3 Launching

2.3.1 Basketball_P.c_launch1

After picking up the ball, the plate tilts clockwise slightly before launching (Fig 2.3), ensuring the ball slides smoothly down the plate.

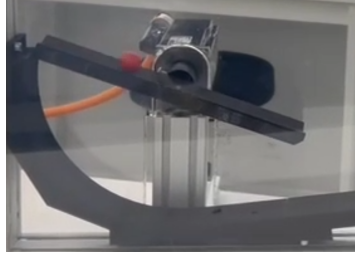


Figure 2.3: Launch Angle 1



Figure 2.4: Launch Angle 2

2.3.2 Basketball_P.accelTime

This parameter controls the acceleration time to prevent the ball from prematurely rolling off the plate.

2.3.3 Basketball_P.c_launch2

This angle determines where the plate stops after launching the ball (Fig 2.4). Adjusting this angle allows us to change where the ball hits the basket. Our goal is to increase shooting accuracy by targeting the red rectangular area and avoiding the yellow and blue parts (Fig 2.5).

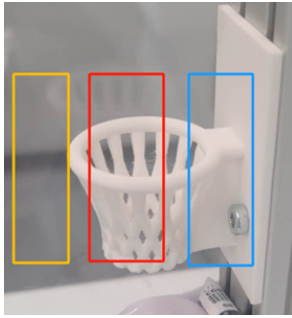


Figure 2.5: Desired Basket Region

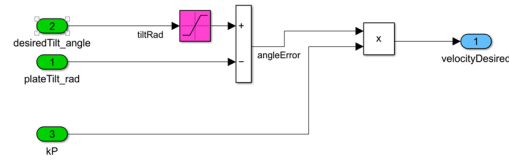


Figure 2.6: Velocity Calculation

2.3.4 Basketball_P.vmax_AM8111_1F21

The system controls the launch velocity by calculating 'velocityDesired' as the product of the angle error between 'desiredTilt_angle' and 'plateTilt_rad' with a large gain 'kP' (Fig 2.6). However, this often exceeds the velocity limits due to the large 'kP' value. Therefore, the actual launch velocity is constrained by these limits, specifically adjusted through 'vmax_AM8111_1F21'. This approach allows effective control over the launch velocity despite the high 'kP' value.

3 Stabilization

3.1 Process Overview

To fully understand the process, let us review the entire sequence of operations.

First, activate the "activateTiltControl" to enable all subsequent operations involving the rotation of the plate. Upon clicking "triggerPickup", the plate rotates clockwise to a nearly horizontal position, lifting the ball from the base. Next, clicking "triggerLaunch" causes the plate to tilt downwards clockwise, allowing the ball to slide naturally under gravity from the left end of the plate to the right. After a predetermined time, the plate is rapidly tilted back counterclockwise to launch the ball.

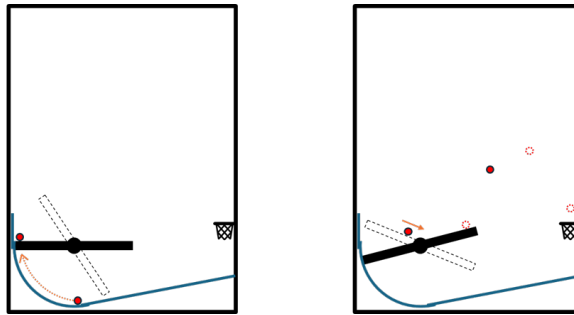


Figure 3.1: Pickup and Launch process

3.2 Ball Position Stabilization

In the development of our automated ball shooting system, we identified a critical issue related to the stabilization of the ball's initial position after it was retrieved by the flat plate. Minor deviations in the ball's initial positioning resulted in inconsistent projectile trajectories, occasionally preventing successful launches.

Upon careful observation, it was noted that the plate exhibited a triangular concavity at its center. Given the spherical nature of the ball, it naturally gravitates towards the center of this concavity during its sliding motion, providing a degree of stabilization. Leveraging this inherent tendency, we implemented a mechanism to ensure a more reliable initial position for the ball.



Figure 3.2: Triangular Concavity in Plate

The procedure begins with the ball being picked up by the flat plate and allowed to slide for a short distance, thus moving towards and stabilizing in the concave center. Following this, the plate is

gradually rotated counterclockwise. This controlled rotation causes the ball to come to rest at the left end of the plate, securely positioned against the left wall. This method establishes a relatively stable and repeatable initial position for the ball.



Figure 3.3: Ball Position Stabilization

By employing this stabilization mechanism, we can more precisely control other launch parameters, thereby ensuring that the ball follows the intended trajectory. This approach significantly increases the probability of the ball successfully entering the basket, thereby enhancing the overall accuracy and reliability of our automated ball shooting system.

3.3 Ideas About Automation

Since the ball's initial position is stabilized, each manually triggered launch follows a highly consistent trajectory. Consequently, we considered integrating the pickup and launch processes into a single automated sequence. This integration implies that activating "triggerPickup" will automatically proceed through the stabilization and launch phases without the need for a separate "triggerLaunch" activation.

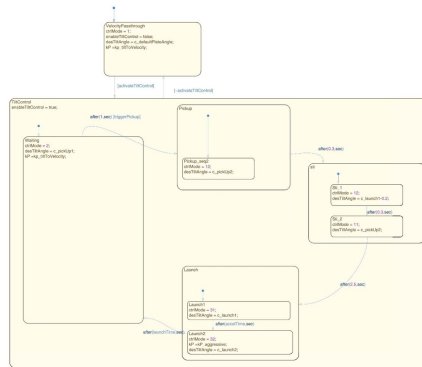


Figure 3.4: Simulink Diagram

However, requiring manual activation of "triggerPickup" each time still falls short of our goal. Our next step involves incorporating infrared break beam sensors and piezoelectric sensors to automate the pickup process and to count successful shots. This advancement will further streamline the operation of our system and improve its overall efficiency and effectiveness.

4 Sensor Configuration

This chapter covers the selection, installation, and configuration of the piezoelectric sensor and the infrared breakbeam sensor used in the basketball shooting project. The piezoelectric sensor is employed to detect successful basketball shots by sensing impacts on the hoop, thereby recording the number of successful shots. The infrared breakbeam sensor detects the position of the basketball, facilitating automatic ball retrieval and also counting the number of shots.

4.1 Piezoelectric Sensor

A piezoelectric sensor generates an electric charge under mechanical stress, making it ideal for detecting impacts and vibrations. In this basketball project, we installed the sensor below the hoop to detect the basketball's impact, thereby counting each shot.

4.1.1 Sensor Selection and Installation

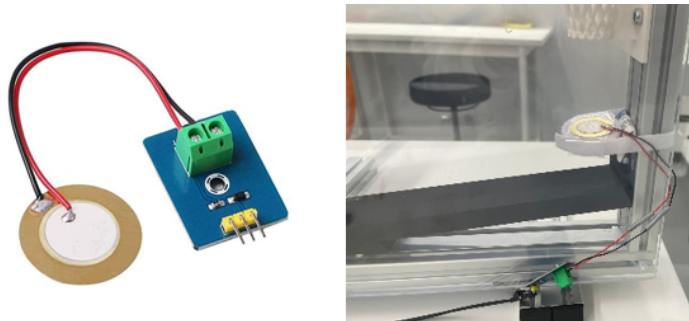


Figure 4.1: Piezoelectric Sensor and Installation Image

Sensor Selection:

The DollaTek Analog Ceramic Piezo Vibration Sensor Module was chosen for this project. This sensor operates at 3.3V or 5V with a working current of 1mA and features an analog output interface. It buffers the response of the piezoelectric converter to stress changes, generating a measurable output voltage change proportional to the intensity of the vibration.

Installation Location:

The sensor was initially installed below the hoop on the track, but vibrations from missed shots caused significant voltage changes due to its high sensitivity. To address this, the sensor was moved to a middle position between the hoop and the track, secured with brackets and tape. This new position ensured accurate detection of successful shots while minimizing false readings from missed shots.

4.1.2 Connection to device

Input:

In MATLAB Simulink, we created a model to supply the power to the device. We set the input

value to 16383 and converted it to int16 to match the device’s data type, with the output named "analogueOut_EL4004_Ch2."

In Visual Studio, we configured this voltage value connected to "AO (analog output) Channel 1" designated as "Pressuresensor_power," which outputs a stable 5V to power the piezoelectric sensor, ensuring its proper operation and effective signal detection.

Output:

The sensor’s output pin is connected via wire to "AI (analog input) Standard Channel 1". As shown in Figure 4.2, this channel is named "Pressuresensor_voltage". This setup ensures the sensor’s output voltage is correctly read and monitored.

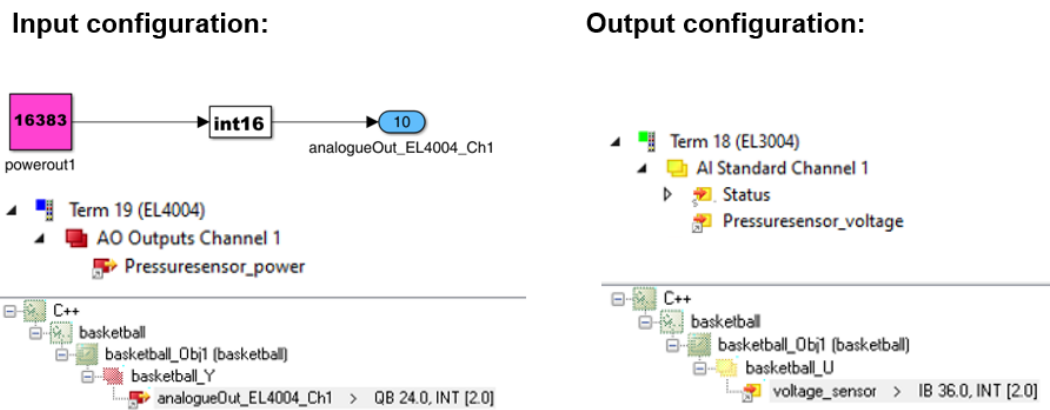


Figure 4.2: Piezoelectric Sensor connection to device

4.1.3 Signal Processing

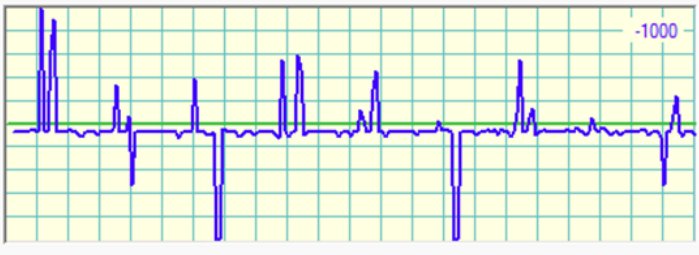


Figure 4.3: Piezoelectric Sensor signal

Data Monitoring and Threshold Setting:

By real-time monitoring of the voltage data output by the piezoelectric sensor, we discovered that when the basketball hits the hoop and strikes the sensor, a significant positive peak signal is generated. As shown in the figure 4.3, when the ball hits the sensor, the voltage signal rises significantly.

Based on the actual shooting effect, we can set a threshold. When the monitored voltage exceeds this threshold, it is considered a successful shot. In this project, we set the threshold to 150. When the voltage signal exceeds -150, a successful shot is recorded.

Signal Processing Code:

To calculate the basketball goals, we wrote PLC codes to process the voltage signal output by the sensor. The precise codes and logic are shown in figure 4.4:

When a voltage signal exceeding 150 is detected, we record a successful shot and start a timer to prevent multiple recordings of the same shot.

```

// calculate goal
timer(IN := timerStart, PT := elapsedTime);

CASE state OF
0:
  IF act_voltagesensor > -150 THEN
    IF NOT timerStarted THEN
      act_goal := act_goal + 1;
      timerStart := TRUE;
      timerStarted := TRUE;
      state := 1;
    END_IF
  END_IF
1:
  IF timer.Q THEN
    timerDone := TRUE;
  END_IF
  IF timerDone THEN
    timerStart := FALSE;
    timerDone := FALSE;
    timerStarted := FALSE;
    state := 0;
  END_IF
END_CASE

```

Initial State (state = 0):

- When the output voltage of the sensor exceeds the set threshold of -150, check if *timerStarted* is FALSE.
- If *timerStarted* is FALSE, it indicates a new valid shot. Increase the successful shot count and restart the timer
- Set *timerStarted* to TRUE and change the state to 1.

Timing State (state = 1):

- When the timer reaches the preset time (*timer.Q* is TRUE), set *timerDone* to TRUE.
- Check if *timerDone* is TRUE, if so, reset all timer-related variables (*timerStart*, *timerDone*, *timerStarted*) and change the state to 0, waiting for the next shot signal.

Figure 4.4: Code and Logic for using Piezoelectric sensor

4.2 Infrared Breakbeam Sensor

The infrared breakbeam sensor uses an infrared emitter and receiver to detect objects passing between them. This characteristic makes it highly suitable for detecting objects crossing a specific path. In this basketball shooting project, we installed an infrared breakbeam sensor to detect basketballs passing through the hoop, thereby counting each shot.

4.2.1 Sensor Selection and Installation

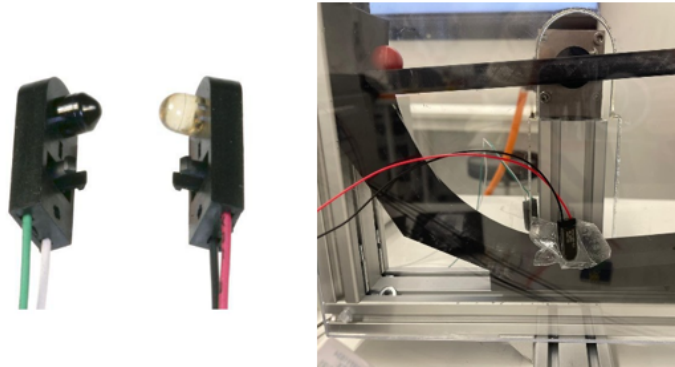


Figure 4.5: Infrared Breakbeam Sensor and Installation Image

Sensor Selection:

We selected the Optek OPB100Z infrared breakbeam sensor module, which consists of an infrared LED (emitter) and a silicon phototransistor (receiver). This sensor has the following key features:

- Maximum detection distance: exceeds 36 inches (91.4 cm), fully meeting the requirement
- Input voltage for emitter: 1.7V, for the typical forward voltage of the infrared LED
- Input voltage for receiver: 5V, ensuring its normal operation

Installation Location:

The sensor is installed at the lowest point of the orbit. The emitter and receiver are located on the different side of orbit, facing to each other. Once the ball rolls down the orbit and pass through the lowest point, the sensor will record it and the number of shots will increase by 1.

4.2.2 Connection to device



Figure 4.6: Infrared Breakbeam sensor input configuration

Input:

As the input configuration of the piezoelectric sensor, we first set up the model in Simulink and then connected it to the output channels in Visual Studio. As shown in Figure 4.6, we supplied two stable power sources: 1.7V to "AO Outputs Channel 2" for the emitter and 5V to "AO Outputs Channel 3" for the receiver.



Figure 4.7: Infrared Breakbeam sensor output configuration

Output:

Since we cannot directly measure the sensor's current, we added a resistor to measure the voltage changes instead. The schematic and installation are shown in Figure 4.7. After installation, we linked the output signal to "AI Outputs Channel 3," labeled "Infrared_voltage". Then we can monitor the signal precisely.

4.2.3 Signal Processing

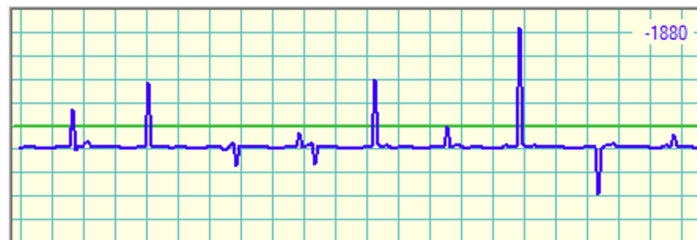


Figure 4.8: Infrared Breakbeam sensor signal

Data Monitoring and Threshold Setting:

By real-time monitoring the voltage output from the infrared breakbeam sensor, we observed a distinct positive peak when the ball passes the sensor, as shown in Figure 4.8. Based on these observations, we set a threshold to detect even small peaks. If the monitored voltage exceeds this threshold ($\text{act.infraredsensor} \geq 10$), it indicates a basketball shot attempt.

Signal Processing Code:

To record the total number of basketball shot attempts, we wrote PLC codes to process the voltage signals output by the sensor. The detailed code and logic are shown in Figure 4.9:

```
// calculate pick up
timer1(IN := timerStart1, PT := elapsedTIme1);

IF act_infraredsensor > 10 THEN
    gui_triggerPickup := TRUE;
END_IF

CASE statel OF
    0:
        IF gui_triggerPickup THEN
            IF NOT timerStarted1 THEN

                timerStart1 := TRUE;
                timerStarted1 := TRUE;
                statel := 1;
            END_IF
        END_IF
    1:
        IF timer1.Q THEN
            timerDone1 := TRUE;
            act_pickup := act_pickup + 1;
        END_IF
        IF timerDone1 THEN
            timerStart1 := FALSE;
            timerDone1 := FALSE;
            timerStarted1 := FALSE;
            statel := 0;
        END_IF
    END_CASE

IF (gui_activateTiltControl=TRUE) AND (prevbutton = FALSE) THEN
    act_goal := 0;
    act_pickup := 0;
END_IF
```

Voltage Threshold Check:

If *act_infraredsensor* exceeds 10, set *gui_triggerPickup* to TRUE.
(start shooting, from pick up to launch)

State 0:

- If *gui_triggerPickup* is TRUE and *timerStarted1* is FALSE:
Indicate a new valid shot and start the *timer1*
Change the state (*statel*) to 1

State 1:

- If *timer1.Q* is TRUE (the timer reaches the preset time) :
Set *timerDone1* to TRUE
Increase the successful pickup count (*act_pickup*) by 1.
- If *timerDone1* is TRUE:
Reset the timer and state variables
Change state (*statel*) to 0

Tilt Control Reset:

- **Reset counters:**
If *gui_activateTiltControl* is TRUE and *prevbutton* is FALSE:
(every time when we push start button)
Reset *act_goal* and *act_pickup* to 0.
- **Update Previous Button State:**
Set *prevbutton* to the current state of *gui_activateTiltControl*.

Figure 4.9: Code and Logic for using Infrared Breakbeam sensor

When the sensor voltage exceeds the threshold of 10, the shooting process begins, marking a valid detection and starting a timer to prevent duplicate recordings. Once the timer finishes, it increments the successful detection count and resets all states. If tilt control is activated, all counters are reset.

5 Results and Discussion

Above two sensors were described which we added in order to count the number of successful baskets.

After adding a piezoelectric sensor we ran into a problem of the contact area of the sensor being not big enough and sometimes it could not detect all successful baskets. To avoid it we had to place the sensor a little bit closer to the basket (see picture below 5.1), it could improve the system and the sensor detected more baskets, but unfortunately, it did not solve the problem completely. As an improvement, the sensor should be attached properly either to the wall or to the bottom part of the construction. It also requires finding an optimal position for the sensor to detect all hits and make them more controllable.

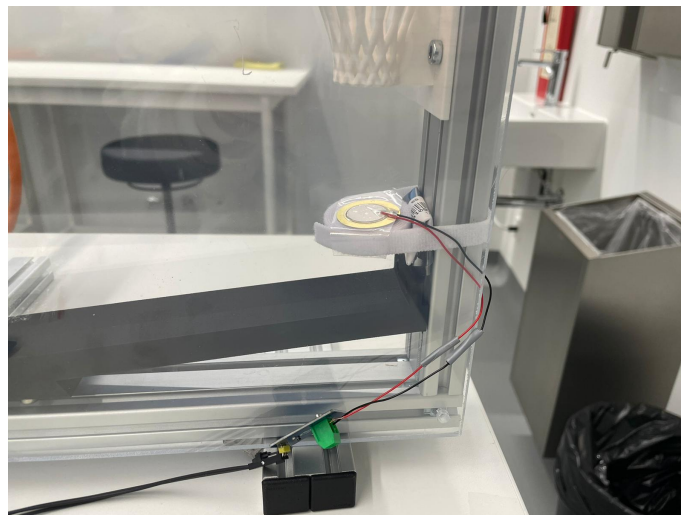


Figure 5.1: Piezoelectric sensor

The infrared sensor worked very precisely and could detect the ball every time. The sensor could measure objects on a distance up to 91.4 cm which was not necessary in our case. In our project, it was necessary just to check for 2 cm. It could be a small improvement for the project - to have the sensor with a more suitable detection distance to avoid some potential mistakes. Such a sensor is also cheaper.

At the end of the project 2 different tests were done in order to check the efficiency of the controller:

1. Scoring baskets within 2 minutes
2. Scoring as many baskets as possible with 10 shots

In general, the controller demonstrated very good precision and consistency in the whole system and equipment.

The first test showed good results. Within 2 minutes 22 baskets were scored and just 3 were missed.

With the second test 10 out of 10 baskets were scored.

The first test showed that the controller was not precise enough for a 100% result. The controller requires some further investigation to understand the issue better and answer the question of why the pendulum throws the ball differently from time to time. To solve this problem it would be useful to configure a closed-loop system instead of an open-loop, which we have right now. Closed-loop system will analyze the position of the ball and depending on this information it will behave differently. For example, the pendulum can wait for the ball if it does not arrive at the place where it should be picked up by the pendulum. Such change might make the system more stable.

In conclusion, I would say that the achieved results can be considered a success, however, the system can be improved by adding different sensors and fine-tuning the parameters.